When is Generated Code Difficult to Comprehend? Assessing AI Agent Python Code Proficiency in the Wild

NANTHIT TEMKULKIAT*, CHAIYONG RAGKHITWETSAGUL*, and MORAKOT CHOETKIERTIKUL, Faculty of ICT, Mahidol University, Thailand
RUKSIT ROJPAISARNKIT, Nara Institute of Science and Technology, Japan
RAULA GAIKOVINA KULA, The University of Osaka, Japan

The rapid adoption of AI coding agents is fundamentally shifting software developers' roles from code authors to code reviewers. While developers spend a significant portion of their time reading and comprehending code, the linguistic proficiency and complexity of the Python code generated by these agents remain largely unexplored. This study investigates the code proficiency of AI agents to determine the skill level required for developers to maintain their code. Leveraging the AIDev dataset, we mined 591 pull requests containing 5,027 Python files generated by three distinct AI agents and employed pycefr, a static analysis tool that maps Python constructs to six proficiency levels, ranging from A1 (Basic) to C2 (Mastery), to analyze the code. Our results reveal that: AI agents predominantly generate Basic-level code, with over 90% of constructs falling into the A1 and A2 categories, and less than 1% classified as Mastery (C2); AI agents' and humans' pull requests share a broadly similar proficiency profile; High-proficiency code by AI agents are from feature addition and bug fixing tasks. These findings suggest that while AI-generated code is generally accessible to developers with basic Python skills, specific tasks may require advanced proficiency to review and maintain complex, agent-generated constructs.

CCS Concepts: • **Software and its engineering → Collaboration in software development**.

Additional Key Words and Phrases: Python, Proficiency, AI coding agents

ACM Reference Format:
Nanthit Temkulkiat, Chaiyong Ragkhitwetsagul, Morakot Choetkiertikul, Ruksit Rojpaisarnkit, and Raula Gaikovina Kula. 2026. When is Generated Code Difficult to Comprehend? Assessing AI Agent Python Code Proficiency in the Wild . In *23rd International Conference on Mining Software Repositories (MSR '26), April 13–14, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3793302.3793580

1 Introduction

AI coding agents are reshaping software development. With the rise of autonomous AI coding agents such as GitHub Copilot and Cursor, developers and AI agents are mutually contributing code to software projects. Recent empirical findings [5] report that OpenAI's Codex generated over 400,000 pull requests (PRs) across open-source GitHub repositories within less than two months of its release in May 2025. GitHub also reports that nearly 80% of new developers use GitHub Copilot in their first week [4].

*Both authors contributed equally to this research.

Authors' Contact Information: Nanthit Temkulkiat; Chaiyong Ragkhitwetsagul; Morakot Choetkiertikul, Faculty of ICT, Mahidol University, Salaya, Nakhon Pathom, Thailand; Ruksit Rojpaisarnkit, Nara Institute of Science and Technology, Nara, Japan; Raula Gaikovina Kula, The University of Osaka, Osaka, Japan.

This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
Manuscript submitted to ACM

```
52  -     results = forecast.simulate_years(secs, horizon)
52  +     trajectory = forecast.forecast_disruptions(
53  +         secs,
54  +         horizon,
55  +         curve=curve,
56  +         pop_size=pop_size,
57  +         generations=generations,
58  +     )
59  +     results = [forecast.ForecastPoint(t.year, t.capability,
      [s for s in t.sectors if s.disrupted]) for t in trajectory]
```

Fig. 1. A code change made by OpenAI Codex using a list comprehension with an `if` statement (line 59)

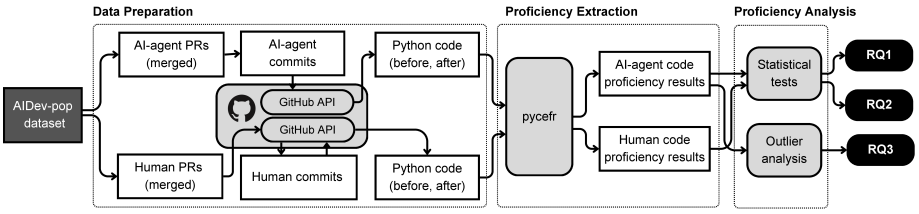

Fig. 2. The overview of this study

The role of software developers is dramatically shifting. With the help of AI coding agents, developers not only review code written by humans but also AI-agents [10, 11]. Reading and comprehending code needs a lot of time and mental energy. A study by Minelli et al. [6] shows that developers spend 70% of their time reading and understanding code. Given the breakneck pace of code generation by AI coding agents, the time required to understand code may increase, and reading and understanding AI-generated code has become a key skill for today's software developers.

Due to the steep increase in AI-enabled projects, the number of software projects written in Python is also sharply increasing [4]. Python language offers many ways to write code, and a developer's proficiency is essential for using specific language idioms, i.e., *Pythonic* code [1]. Python proficiency is an essential factor in open-source software projects, where contributors need to understand the existing code, especially for newcomers.

Nonetheless, a code change made by an AI agent may contain complex code constructs that require Python proficiency to understand correctly. Figure 1 shows a code change in AGI-Alpha-Agent-v0 project[1] that an AI agent committed to add a new feature. Line 59 of the added code contains a nested list comprehension with an `if` statement. Although this code is compact and concise, it is more difficult to understand than its `for` loop counterpart. Thus, maintaining this code needs a highly proficient contributor.

While prior research has explored how external factors like prompt English proficiency influence the level of AI-generated code [9], there is currently a lack of research directly comparing the proficiency of AI-contributed code against human-written code in real-world software projects. Thus, this study fills in the gap. Leveraging the large-scale Python code in PRs created by AI coding agents from the AIDev dataset [5], we analyzed and extracted the coding proficiency

[1]https://github.com/MontrealAI/AGI-Alpha-Agent-v0/pull/774

Table 1. Python proficiency levels in pycefr

Level group	Level and construct examples
A Basic user	A1: Breakthrough or beginner (e.g., `print`, `if` statement, list) A2: Waystage or elementary (e.g., `open` function (files), nested list)
B Independent user	B1: Threshold or intermediate (e.g., list with dictionary, `with`) B2: Vantage or upper intermediate (e.g., list comprehension, __dict__attribute)
C Proficient user	C1: Effective operational proficiency or advanced (e.g., __slots__, generator) C2: Mastery or proficiency (e.g., metaclass, decorator class)

of their contributed code constructs using an automated tool for Python proficiency analysis called pycefr [8]. We aim to answer the following research questions in this study.

- RQ1: *What are the Python proficiency levels of AI agents' code?*
- RQ2: *What are the differences between the Python proficiency levels of AI agents' and humans' written code?*
- RQ3: *What kinds of AI agents' pull requests contain the most proficient code?*

To the best of our knowledge, this is the first study to compare the coding proficiency of AI coding agents with human-written code and paves the way for future studies in this area. A replication package is made publicly available[2].

2 Background

2.1 CEFR and Python Code Proficiency Analysis

The Common European Framework of Reference for Languages (CEFR) [7] constitutes an internationally standardized framework for assessing linguistic proficiency. It classifies language skills into six levels: A1 (breakthrough) and A2 (waystage) for basic users; B1 (threshold) and B2 (vantage) for independent users; and C1 (advanced) and C2 (mastery) for proficient users. By providing this unified metric, the framework streamlines the evaluation of competence for educators and learners alike. Furthermore, the CEFR enables the alignment of national qualifications and facilitates skill verification for academic institutions and employers.

Inspired by CEFR, pycefr [8] is a static analysis tool that automates the assessment of code proficiency in Python projects. Given Python code, pycefr identifies the code constructs used and estimates the skill level required to understand them. By adapting the CEFR's linguistic levels to the domain of programming, the tool maps Python code constructs to six distinct levels of complexity (see Table 1). For instance, fundamental constructs such as `if` statements and nested lists are categorized as Basic (A1 and A2, respectively). `break` statements and list comprehensions are mapped to the Intermediate levels (B1 and B2), while advanced features, including generator functions and metaclasses, correspond to the Proficient levels (C1 and C2). The complete list of code constructs and their CEFR levels can be found in the pycefr

[2]Replication package: https://github.com/EarnGH/AIDevMSR2026

Table 2. Summary of the collected and analyzed data for RQ1

Category	PRs	Repos	Commits	Files
Collected merged PRs	591	145	1,830	5,027
Analyzed by pycefr	524	139	1,463	3,441

repository [2]. It is used in a recent study to analyze the progression of proficiency in Python OSS projects [3]. In this study, we employ pycefr to analyze the proficiency of AI agents' code and compare it with that of humans.

3 Study Design

The overview of the study is depicted in Figure 2. We use the AIDev dataset [5], a large-scale dataset of AI agent PRs, and select the AIDev-pop subset, which includes only repositories with more than 100 stars, focusing on popular repositories (filtered by stars), and excludes most toy projects.

3.1 Data Preparation

In this study, we focus exclusively on Python repositories and analyze code changes extracted from PR-level commits for both AI agents and human developers. Thus, we first extracted the PR, repository, and commit-related information required for code-level analysis. We then associated each PR with its corresponding project and programming language, and further merged commit-level information to obtain detailed file-level changes linked to each PR. After filtering for only Python projects, we filtered for merged PRs, as they represent the code changes that were accepted. Lastly, for the commits, we filtered only the commits in which AI coding agents were the authors (e.g., `devin-ai-integration[bot]`). Nonetheless, Claude Code and OpenAI Codex attribute their contributions to human developers [5]. Thus, the authors of the commits using them were varied, and we could not precisely decide which commits were actually made by agents. To ensure the validity of our analysis, we restricted ourselves to analyzing only the three remaining agents: Copilot, Cursor, and Devin.

As shown in Table 2 (Collected merged PRs section), this resulted in 591 PRs from 145 Python projects, containing 1,830 commits and associated with 5,027 Python files. Next, we retrieved the added code in each commit. pycefr only works on complete Python files. Thus, for each Python file, we extracted the patch information from the commit details (i.e., line-level diffs containing code additions and deletions) and queried the GitHub API to retrieve the file's version before the commit to apply the diffs. Using this information, we reconstructed both the *before* and *after* versions of each file. Files introduced for the first time in a PR were identified as newly created files and were represented by a single (after) version. For human code, we filtered the merged PRs and queried the GitHub API to obtain the associated commits and the code changes. Finally, we retrieved 785 PRs from 384 Python repositories, containing 3,548 commits and covering 20,063 Python source code files.

3.2 Proficiency Extraction

We applied pycefr to Python source code files, both agent-generated and human-written. pycefr failed to analyze some files due to parsing issues with the abstract syntax tree. Thus, the total number of Python code files analyzed successfully by pycefr was slightly lower than the ones we initially retrieved. As displayed in Table 2 (Analyzed by pycefr section), for AI agents, there were 3,441 Python files associated with 1,463 commits across 524 PRs in 139 repositories. For

Table 3. Distribution of AI agents' Python code constructs classified into proficiency levels. The highest values are highlighted.

Agent	A1	A2	B1	B2	C1	C2
All agents	91,223 (35.72%)	138,894 (54.39%)	12,639 (4.95%)	8,913 (3.49%)	2,731 (1.07%)	972 (0.38%)
Copilot	**51,807 (36.85%)**	75,857 (53.96%)	6,614 (4.70%)	4,080 (2.90%)	**1,655 (1.18%)**	561 (0.40%)
Cursor	15,959 (31.76%)	**29,449 (58.61%)**	2,337 (4.65%)	1,755 (3.49%)	521 (1.04%)	**221 (0.44%)**
Devin	23,467 (36.35%)	33,588 (52.02%)	**3,688 (5.71%)**	**3,078 (4.77%)**	555 (0.86%)	190 (0.29%)

humans, there were 12,366 Python files associated with 2,814 commits across 694 PRs in 151 repositories successfully analyzed.

For newly added files, all detected constructs and their associated proficiency levels were counted directly. For modified files with both before and after versions, we performed a differential analysis by matching constructs between the two versions and computing the difference in their occurrence counts. Only constructs newly introduced in the later version were retained, while negative differences were clipped at zero, similar to the previous study [3]. This procedure ensures that the proficiency analysis reflects only the code introduced in a given PR, rather than the entire file content, which may include contributions from prior commits authored by human developers or other agents. Moreover, we focus only on the added code and ignore the deleted code by AI agents, as that is what the human contributors need to maintain.

3.3 Proficiency Analysis

Methodology to answer RQ1. We aggregated proficiency results, i.e., the number of code constructs classified into one of the six Python proficiency levels, across all AI coding agents to examine their distribution. Then, we divided the proficiency scores of each AI agent and compared the differences among them.

Methodology to answer RQ2. We filtered the repositories to include only those in which both AI agents and human programmers contributed code. Then, we compared the distributions of code constructs across Python proficiency levels in AI-generated and human-written code.

Methodology to answer RQ3. We identified PRs in which AI agents introduce unusually high amounts of proficient-level Python constructs. We aggregated proficiency scores at the PR level by summing the counts of C1 and C2 constructs introduced by AI agents across all files and commits in each PR. We then detected PRs with exceptionally large C1 + C2 totals using the IQR-based outlier criterion ($> Q_3 + 1.5 \times IQR$). Using these outliers, we extracted 11 task categories (e.g., `feat`, `fix`, `docs`) associated with the PRs from the AIDev dataset.

4 Results

4.1 RQ1: What are the Python proficiency levels of AI agents' code?

4.1.1 Overall Proficiency of AI Coding Agents. Table 3 (first row) summarizes the number of Python code constructs by the CEFR-based proficiency levels for all the agents. The results show that the majority of AI-generated Python code constructs fall within the Basic proficiency range. In particular, A2 (Waystage) is the most dominant level, accounting for 54.39% of all code constructs, followed by A1 (Breakthrough) (35.723%). Together, Basic-level constructs (A1 and A2) comprise more than 90% of the AI agents' contributed code. Intermediate-level constructs appear far less frequently. B1

(Threshold) accounts for 4.95%, while B2 (Vantage) represents only 3.49%. Proficient-level constructs are relatively rare: C1 (Advanced) and C2 (Mastery) contribute 1.07% and less than 0.5%.

4.1.2 Proficiency of Each AI Coding Agent. Table 3 presents the proportion of CEFR levels for code generated by Copilot, Cursor, and Devin. We can clearly see that Copilot is the most widely adopted agent among the three agents, with the number of analyzed code constructs far exceeding those of the other agents. Across all agents, Basic-level constructs (A1 and A2) dominate the generated code, similar to the overall result. For all three agents, A2 is the most prevalent (52.02% to 58.61%). A1 is the second most common level (31.76% to 36.85%). Intermediate-level constructs (B1 and B2) appear less frequently but consistently across agents. B1 ranges from 4.65% to 5.71%, while B2 contributes between 2.90% and 4.77%. Proficient-level constructs are relatively rare for all models. C1 remains below 1.20%, while C2 accounts for less than 1% of the generated code in all cases. Copilot and Cursor show the highest proportion of C1 and C2 constructs (1.18% and 0.44%), respectively.

A chi-square test of independence indicates that proficiency distributions differ statistically across AI agents ($\chi^2 = 1,117.56$, $df = 10$, $p < 0.001$); however, the effect size is small (Cramér's $V = 0.046$), suggesting that the observed differences are minor in magnitude. A rank-based Kruskal–Wallis test yields consistent results ($H = 343.41$, $p < 0.001$).

Answer to RQ1: AI coding agents predominantly generate Python code that relies on beginner and elementary language constructs, with fewer contributions involving intermediate and proficient Python features. There are statistically significant differences in proficiency distributions across agents with a small effect size.

4.2 RQ2: What are the differences between the Python proficiency levels of AI agents' and humans' written code?

After filtering the AIDev dataset for repositories in which both AI agents and human programmers contributed code, we obtained 39 repositories associated with 230 PRs, 647 commits, and 1,527 files for agents and 385 PRs, 1,348 commits, and 6,950 files for humans.

As shown in Table 4, for both AI agents and humans, Basic-level constructs (A1 and A2) dominate the generated code. AI-generated code consists primarily of A2 and A1 constructs, while human-written code exhibits a similar pattern. Together, Basic-level constructs comprise more than 88% of the code in both groups. The Intermediate-level constructs (B1 and B2) appear at comparable frequencies in AI-generated and human-written code. In contrast, Proficient-level constructs (C1 and C2) remain rare for both AI and humans, accounting for less than 2% of total occurrences in each case. Nonetheless, the AI agents generated slightly more C1 code than humans (1.12% vs. 0.85%). On the other hand, human programmers created slightly more C2 code than AI agents (0.57% vs. 0.42%). A chi-square test indicates a statistically significant difference ($\chi^2 = 1,124.52$, $df = 5$, $p < 0.001$) with a small effect size (Cramér's $V = 0.041$), similar to the rank-based Kruskal–Wallis test ($H = 265.59$, $p < 0.001$).

Answer to RQ2: AI-generated and human-written code exhibit statistically significant differences in proficiency distributions. Nonetheless, their overall proficiency profiles are broadly similar. Agents generated slightly more C1 code than humans.

4.3 RQ3: What kinds of AI agents' pull requests contain the most proficient code?

In total, we identify 79 outlier agent PRs with the C1 + C2 counts ranging from 8 to 339, with a median of 19 and an average of 41.41. After analyzing these PRs, we identified seven major tasks. The scatter plot in Figure 3 depicts the seven tasks with their number of PRs and total counts of C1 + C2 (in the log scale). The size of the bubbles represents

Table 4. Proficiency distribution for agents and human code

	A1	A2	B1	B2	C1	C2
AI agents	35.52%	52.85%	5.14%	4.94%	1.12%	0.42%
Human	31.60%	57.70%	5.37%	3.91%	0.85%	0.57%

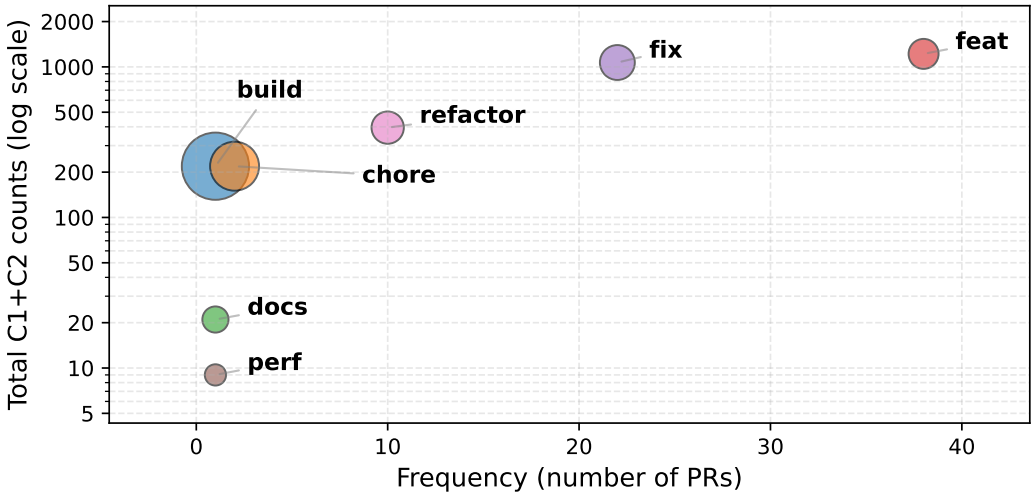

Fig. 3. Tasks of the outlier agent PRs with a high number of proficient code (C1 + C2) (log scale)

the average count of C1 + C2 of each task. The most frequent task is feature development (*feat*) (38 PRs with 1,220 C1 + C2 code constructs), followed by bug remediation (*fix*) (22 PRs, 1,069 constructs), *refactor* (10 PRs, 395 constructs), *chore* (2 PRs, 219 constructs), *build* (1 PR, 219 constructs), *docs* (1 PR, 21 constructs), and performance improvement (*perf*) (1 PR, 9 constructs). Interestingly, the task with the highest average of C1 + C2 is *build* (219 per PR), *chore* (109.50), and *fix* (48.59).

Answer to RQ3: PRs that perform new feature addition contain the highest total number of proficient code. However, PR that relate to building the project contain the highest average number of proficient code per PR.

5 Discussion and Future Work

This study finds that all AI coding agents exhibit a similar distribution of Python code proficiency, with slight differences among the agents and comparable to that of human programmers. This implies that maintaining agents' code may not differ from that of human counterparts. Nonetheless, some of the agents' PRs contain highly proficient code, especially in adding features and fixing bugs. Thus, maintainers need to pay attention to such PRs when assigning reviewers, ensuring the introduced code is reviewed correctly.

Future work includes (1) a deeper investigation of the scenarios and reviews of agents' PRs, and determining whether the high proficiency affected the review process; (2) analyzing proficiency in AI agents' vs. humans' code per task (e.g., feature addition, refactoring) across project types (e.g., AI vs. cryptocurrency); (3) studying standardized agent prompting languages, which might be necessary to ensure consistent code proficiency across an organization; (4) refactoring agents' and humans' code based on proficiency.

References

[1] Carol V. Alexandru, José J. Merchante, Sebastiano Panichella, Sebastian Proksch, Harald C. Gall, and Gregorio Robles. 2018. On the usage of pythonic idioms. In *Onward!'18,*. Association for Computing Machinery, New York, NY, USA, 1–11.

[2] anapgh. 2021. pycefr: Identifying Python3 Code Level Using the CERFL Framework as Inspiration. https://github.com/anapgh/pycefr

[3] Rujiphart Charatvaraphan, Bunradar Chatchaiyadech, Thitirat Sukijprasert, Chaiyong Ragkhitwetsagul, Morakot Choetkiertikul, Raula Gaikovina Kula, Thanwadee Sunetnanta, and Kenichi Matsumoto. 2025. PyGress: Tool for Analyzing the Progression of Code Proficiency in Python OSS Projects. In *ASE'25*. IEEE, Piscataway, NJ, USA.

[4] GitHub. 2025. Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1. https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1

[5] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv preprint arXiv:2507.15003* (2025).

[6] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I Know What You Did Last Summer - An Investigation of How Developers Spend Their Time. In *2015 IEEE 23rd International Conference on Program Comprehension (ICPC'15)*. IEEE, Piscataway, NJ, USA, 25–35. doi:10.1109/ICPC.2015.12

[7] Council of Europe. 2025. Common European Framework of Reference for Languages (CEFR). https://www.coe.int/en/web/common-european-framework-reference-languages

[8] Gregorio Robles, Raula Gaikovina Kula, Chaiyong Ragkhitwetsagul, Tattiya Sakulniwat, Kenichi Matsumoto, and Jesus M. Gonzalez-Barahona. 2022. pycefr: Python competency level through code analysis. In *ICPC '22*. Association for Computing Machinery, New York, NY, USA, 173–177.

[9] Ruksit Rojpaisarnkit, Youmei Fan, Kenichi Matsumoto, and Raula Gaikovina Kula. 5555. How Natural Language Proficiency Shapes GenAI Code for Software Engineering Tasks . *IEEE Software* 01 (Oct. 5555), 1–7. doi:10.1109/MS.2025.3622690

[10] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. 2025. Human-In-The-Loop Software Development Agents . In *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 342–352.

[11] Wannita Takerngsaksiri, Chakkrit Tantithamthavorn, Micheal Fu, Jirat Pasuksmit, Kun Chen, and Ming Wu. 2025. Code Readability in the Age of Large Language Models: An Industrial Case Study from Atlassian. In *ICSME'25*. IEEE, Piscataway, NJ, USA, 732–742.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009